DEPARTMENT OF CSIT
GURU GHASIDAS VISHWAVIDYALAYA, BILASPUR

(A Central University established by the Central Universities Act 2009 No. 25 of 2009)

UG IV SEMESTER

Session: 2025-2026

Subject: Object Oriented Programming with JAVA

Submitted To

Dr. Ratnesh Prasad Srivastava
(Associate Professor)

Submitted By

Prasansha Mishra
BSc(CS),CSIT
Roll No : 24070150

Assignments:

1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

```
import java.io.*;
import java.util.*;
public class FarmerEasy {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        String name[] = new String[10];
        String fruit[] = new String[10];
        String city[] = new String[10];
        int qty[] = new int[10];
        // INPUT
        for(int i=0; i<10; i++) {
            System.out.println("Enter Farmer " + (i+1));
            System.out.print("Name: ");
            name[i] = sc.next();
            System.out.print("Fruit: ");
            fruit[i] = sc.next();
            System.out.print("City: ");
            city[i] = sc.next();
            System.out.print("Quantity: ");
            qty[i] = sc.nextInt();
        }
        // SAVE BEFORE SORT
        FileWriter f1 = new FileWriter("before.txt");
        for(int i=0; i<10; i++) {
```

```
    f1.write(name[i] + " " + fruit[i] + " " + city[i] + " " + qty[i] + "\n");
}
f1.close();
// SORT (simple)
for(int i=0; i<10; i++) {
    for(int j=i+1; j<10; j++) {
        if(qty[i] > qty[j]) {
            int t = qty[i];
            qty[i] = qty[j];
            qty[j] = t;
            String s;
            s = name[i]; name[i] = name[j]; name[j] = s;
            s = fruit[i]; fruit[i] = fruit[j]; fruit[j] = s;
            s = city[i]; city[i] = city[j]; city[j] = s;
        }
    }
}
// SAVE AFTER SORT
FileWriter f2 = new FileWriter("after.txt");
for(int i=0; i<10; i++) {
    f2.write(name[i] + " " + fruit[i] + " " + city[i] + " " + qty[i] + "\n");
}
f2.close();
// SEARCH
System.out.print("Enter name to search: ");
String x = sc.next();
for(int i=0; i<10; i++) {
    if(name[i].equals(x)) {
        System.out.println("Found: " + name[i] + " " + fruit[i] + " " + city[i] + " " + qty[i]);
```

```
            }
        }
        sc.close();
    }
}
```

Output :

Enter Farmer 1

Name: Rahul

Fruit: Mango

City: Delhi

Quantity: 500

Enter Farmer 2

Name: Amit

Fruit: Apple

City: Mumbai

Quantity: 300

Enter Farmer 3

Name: Sita

Fruit: Papaya

City: Bhopal

Quantity: 700

Enter Farmer 4

Name: Riya

Fruit: Mango

City: Indore

Quantity: 200

Enter Farmer 5

Name: Mohan

Fruit: Apple

City: Pune

Quantity: 900

Enter Farmer 6

Name: Neha

Fruit: Mango

City: Jaipur

Quantity: 100

Enter Farmer 7

Name: Karan

Fruit: Papaya

City: Agra

Quantity: 600

Enter Farmer 8

Name: Pooja

Fruit: Apple

City: Delhi

Quantity: 400

Enter Farmer 9

Name: Vivek

Fruit: Mango

City: Lucknow

Quantity: 800

Enter Farmer 10

Name: Ankit

Fruit: Papaya

City: Patna

Quantity: 250

Enter name to search: Riya

Found: Riya Mango Indore 200

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```
import java.util.*;
public class FruitEasy {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String fruit[] = {"Mango", "Apple", "Papaya"};
        int qty[] = new int[3];
        String quality[] = new String[3];
        // INPUT
        for (int i = 0; i < 3; i++) {
            System.out.println("Enter details for " + fruit[i]);
            System.out.print("Quantity: ");
            qty[i] = sc.nextInt();
            // CONDITION: if quantity > 1000 → half
            if (qty[i] > 1000) {
                qty[i] = qty[i] / 2;
```

```
            }
            System.out.print("Quality (good/bad/average): ");
            quality[i] = sc.next();
        }
        // DISPLAY DATA
        System.out.println("\nStored Data:");
        for (int i = 0; i < 3; i++) {
            System.out.println(fruit[i] + " " + qty[i] + " " + quality[i]);
        }
        // SEARCH
        System.out.print("\nEnter fruit to search: ");
        String f = sc.next();
        System.out.print("Enter quality to search: ");
        String q = sc.next();
        boolean found = false;
        for (int i = 0; i < 3; i++) {
            if (fruit[i].equalsIgnoreCase(f) && quality[i].equalsIgnoreCase(q)) {
                System.out.println("Found: " + fruit[i] + " " + qty[i] + " " + quality[i]);
                found = true;
            }
        }
        if (!found) {
            System.out.println("Not Found");
        }
        sc.close();
    }
}
```

Output :

Enter details for Mango

Quantity: 1500

Quality (good/bad/average): good

Enter details for Apple

Quantity: 800

Quality (good/bad/average): average

Enter details for Papaya

Quantity: 1200

Quality (good/bad/average): bad

Stored Data:

Mango 750 good

Apple 800 average

Papaya 600 bad

Enter fruit to search: Mango

Enter quality to search: good

Found: Mango 750 good

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively . The available orchards in a tree-view . Search for a given fruit in the available orchards after taking input from the user & print the tree.

```
import java.util.*;
public class OrchardTree {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String orchard[] = {"A", "B", "C", "D"};
        String fruit[] = {"Apple", "Litchi", "Mango", "Apple"};
        String quality[] = {"good", "good", "good", "bad"};
        // PRINT TREE
        System.out.println("Orchard Tree:");
        for (int i = 0; i < 4; i++) {
            System.out.println("Orchard " + orchard[i]);
            System.out.println("   |-- Fruit: " + fruit[i]);
            System.out.println("   |-- Quality: " + quality[i]);
        }
        // SEARCH
        System.out.print("\nEnter fruit to search: ");
        String search = sc.next();
        boolean found = false;
        System.out.println("\nSearch Result (Tree Form):");
        for (int i = 0; i < 4; i++) {
            if (fruit[i].equalsIgnoreCase(search)) {
                System.out.println("Orchard " + orchard[i]);
                System.out.println("   |-- Fruit: " + fruit[i]);
                System.out.println("   |-- Quality: " + quality[i]);
                found = true;
```

```
            }
        }
        if (!found) {
            System.out.println("Fruit not found in any orchard");
        }
        sc.close();
    }
}
```

Output :

```
Orchard Tree:
Orchard A
   |-- Fruit: Apple
   |-- Quality: good

Orchard B
   |-- Fruit: Litchi
   |-- Quality: good
Orchard C
   |-- Fruit: Mango
   |-- Quality: good
Orchard D
   |-- Fruit: Apple
   |-- Quality: bad

Enter fruit to search: Apple

Search Result (Tree Form):
Orchard A
   |-- Fruit: Apple
   |-- Quality: good
```

Orchard D

|-- Fruit: Apple

|-- Quality: bad

4. Take a map provided as a graph & display all the subtrees of connected points.

```
import java.util.*;
public class GraphSubtrees {
    static int n;
    static int graph[][] = new int[10][10];
    static boolean visited[] = new boolean[10];
    // DFS function
    static void dfs(int v) {
        System.out.print(v + " ");
        visited[v] = true;
        for (int i = 0; i < n; i++) {
            if (graph[v][i] == 1 && !visited[i]) {
                dfs(i);
            }
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        n = sc.nextInt();
        System.out.println("Enter adjacency matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = sc.nextInt();
```

```
            }
        }
        System.out.println("\nSubtrees (Connected Components):");
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                dfs(i);   // print one subtree
                System.out.println();
            }
        }
        sc.close();
    }
}
```

Output :

```
Enter number of nodes: 5

Enter adjacency matrix:
0 1 0 0 0
1 0 0 0 0
0 0 0 1 1
0 0 1 0 1
0 0 1 1 0

Subtrees (Connected Components):
0 1
2 3 4
```

5. Enter entry point & goal point & produce the matching subtree of the graph.

```
import java.util.*;
public class GraphPath {
    static int n;
    static int graph[][] = new int[10][10];
    static boolean visited[] = new boolean[10];
    static boolean found = false;
    // DFS to find path
    static void dfs(int v, int goal) {
        visited[v] = true;
        System.out.print(v + " ");
        if (v == goal) {
            found = true;
            return;
        }
        for (int i = 0; i < n; i++) {
            if (graph[v][i] == 1 && !visited[i] && !found) {
                dfs(i, goal);
            }
        }
        if (!found) {
            System.out.print(""); // do nothing
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        n = sc.nextInt();
```

```
        System.out.println("Enter adjacency matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = sc.nextInt();
            }
        }
        System.out.print("Enter entry point: ");
        int start = sc.nextInt();
        System.out.print("Enter goal point: ");
        int goal = sc.nextInt();
        System.out.println("\nMatching Subtree (Path):");
        dfs(start, goal);
        if (!found) {
            System.out.println("\nNo path found");
        }
        sc.close();
    }
}
```

Output :

```
Enter number of nodes: 5

Enter adjacency matrix:
0 1 0 0 0
1 0 0 0 0
0 0 0 1 1
0 0 1 0 1
0 0 1 1 0

Enter entry point: 2
```

Enter goal point: 4

Matching Subtree (Path):

2 3 4

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
import java.util.*;
public class ShortestPathEasy {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();
        int graph[][] = new int[n][n];
        System.out.println("Enter weight matrix (0 if no edge):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = sc.nextInt();
            }
        }
        System.out.print("Enter start node: ");
        int start = sc.nextInt();
        System.out.print("Enter end node: ");
        int end = sc.nextInt();
        int dist[] = new int[n];
        boolean visited[] = new boolean[n];
        // Initialize
        for (int i = 0; i < n; i++) {
            dist[i] = 9999;   // infinity
```

```
        }
        dist[start] = 0;
        // Dijkstra logic
        for (int count = 0; count < n - 1; count++) {
            int min = 9999, u = -1;
            // find minimum distance node
            for (int i = 0; i < n; i++) {
                if (!visited[i] && dist[i] < min) {
                    min = dist[i];
                    u = i;
                }
            }
            visited[u] = true;
            // update distances
            for (int v = 0; v < n; v++) {
                if (!visited[v] && graph[u][v] != 0 &&
                        dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
        // OUTPUT
        System.out.println("\nShortest distance from " + start + " to " + end + " is: " + dist[end]);
        sc.close();
    }
}
```

Output :

Enter number of nodes: 4

Enter weight matrix:

0 2 0 6

2 0 3 0

0 3 0 1

6 0 1 0

Enter start node: 0

Enter end node: 3

Shortest distance from 0 to 3 is: 6

7. Write a program to demonstrate the use of all keywords made for exception handling.

```
import java.util.*;
public class ExceptionDemo {
   // using throws
   static void checkAge(int age) throws Exception {
      if (age < 18) {
         // using throw
         throw new Exception("Not eligible to vote");
      } else {
         System.out.println("Eligible to vote");
      }
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
```

```
        try {
            // taking input
            System.out.print("Enter a number: ");
            int a = sc.nextInt();
            System.out.print("Enter another number: ");
            int b = sc.nextInt();
            // may cause exception
            int result = a / b;
            System.out.println("Result: " + result);
            // calling function (throws + throw)
            System.out.print("Enter age: ");
            int age = sc.nextInt();
            checkAge(age);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            // always runs
            System.out.println("Program finished (finally block)");
        }
        sc.close();
    }
}
```

Output :

Enter a number: 10

Enter another number: 2

Result: 5

Enter age: 20

Eligible to vote

Program finished (finally block)

8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```
import java.util.*;
// Custom Exception
class NoMoneyException extends Exception {
   NoMoneyException(String msg) {
      super(msg);
   }
}
public class BankWithdraw {
   // method with throws
   static void withdraw(int balance, int amount) throws NoMoneyException {
      if (amount > balance) {
         // throw custom exception
         throw new NoMoneyException("Insufficient Balance!");
      } else {
         balance = balance - amount;
         System.out.println("Withdrawal Successful");
         System.out.println("Remaining Balance: " + balance);
      }
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      try {
         System.out.print("Enter Balance: ");
         int balance = sc.nextInt();
```

```
            System.out.print("Enter Withdraw Amount: ");
            int amount = sc.nextInt();
            withdraw(balance, amount);
        } catch (NoMoneyException e) {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}
```

Output :

Enter Balance: 5000

Enter Withdraw Amount: 2000

Withdrawal Successful

Remaining Balance: 3000

9. Write a program which enables to use a platform by a train one at a time.

```
class Platform {
    synchronized void usePlatform(String name) {
        System.out.println(name + " is using platform");
        try {
            Thread.sleep(1000);
        } catch (Exception e) {}
        System.out.println(name + " left platform");
    }
}
class Train extends Thread {
    Platform p;
    String name;
```

```
    Train(Platform p, String name) {
        this.p = p;
        this.name = name;
    }
    public void run() {
        p.usePlatform(name);
    }
}
public class Main {   //  IMPORTANT
    public static void main(String[] args) {
        Platform p = new Platform();
        Train t1 = new Train(p, "Train A");
        Train t2 = new Train(p, "Train B");
        t1.start();
        t2.start();
    }
}
```

Output :

Train A is using platform

Train A left platform

Train B is using platform

Train B left platform

10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

```
class Restaurant {
   boolean orderReady = false;
   // Waiter gives order
   synchronized void placeOrder(String order) {
      while (orderReady) {
         try { wait(); } catch (Exception e) {}
      }
}
      System.out.println("Waiter placed order: " + order);
      orderReady = true;
      notify(); // notify chef
   }
   // Chef prepares order
   synchronized void prepareOrder() {
      while (!orderReady) {
         try { wait(); } catch (Exception e) {}
      }
      System.out.println("Chef prepared and delivered order");
      orderReady = false;
      notify(); // notify waiter
   }
}
// Waiter Thread
class Waiter extends Thread {
   Restaurant r;
   Waiter(Restaurant r) {
      this.r = r;
```

```
    }
    public void run() {
        r.placeOrder("Pizza");
    }
}
// Chef Thread
class Chef extends Thread {
    Restaurant r;
    Chef(Restaurant r) {
        this.r = r;
    }
    public void run() {
        r.prepareOrder();
    }
}
public class Main {
    public static void main(String[] args) {
        Restaurant r = new Restaurant();
        Waiter w = new Waiter(r);
        Chef c = new Chef(r);
        w.start();
        c.start();
    }
}
```

Output :

Waiter placed order: Pizza

Chef prepared and delivered order